DEV_IMPL_GUIDE - FE-APP-00 Frontend Architecture And Shared UX Foundation

1. Implementation Objective

Implement the shared frontend foundation used by all SOPHIX frontend apps: authentication, runtime configuration, layout shell, API client conventions, localization, theming, error handling, file access, telemetry, and shared UI behavior.

2. Start Here

Read these first:

1. `DD_INDEX.md`
2. `DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0.md`
3. `DD_API-00-API_Catalog_and_Gateway_Standards-v1.0.md`
4. `FOUNDATION_AUTH.md`
5. `FOUNDATION_FILE_STORAGE.md`
6. `FE-APP-00-Frontend_Architecture_and_Shared_UX_Foundation-v1.0.md`

3. Build Order

1. Create the frontend shell and route structure.
2. Add OIDC login/logout/token refresh through `FOUNDATION_AUTH`.
3. Add runtime config loading for operator, locale, theme, feature flags, and API base URLs.
4. Add shared API client behavior: correlation id, idempotency header support, pagination, error mapping, timeout handling.
5. Add role/permission helpers from `EM-CFG-03` and auth claims.
6. Add file upload/download wrappers through `FOUNDATION_FILE_STORAGE`.
7. Add shared loading, empty, stale, partial-error, and retry states.
8. Add telemetry for page load, API latency, failed requests, and client errors.

4. Non-Negotiable Rules

- Frontends call module APIs through the approved gateway/base URL.
- Frontends do not write directly to multiple modules for one command.
- Frontends do not read databases.
- Shared shell can host multiple apps, but business ownership remains with module APIs.
- Cache browser data only where the DD allows it, and show freshness where relevant.

5. Tests

- Auth redirect and token refresh.
- Role-based navigation visibility.
- API error mapping.
- Idempotency header propagation for commands.
- Locale/theme/operator switching.
- File access authorization failure.

6. Shared Frontend Modules To Build

Implement these shared modules before app-specific screens:

Module	Purpose	Minimum implementation
`auth/session`	OIDC session, token refresh, logout, current user.	Stores access token in memory, refreshes before expiry, exposes `user`, `roles`, `operatorCodes`, `accountScope`.
`config/runtime`	Operator, region, theme, language, feature flags, API bases.	Loads config on boot, caches in memory, invalidates on operator switch.
`api/client`	HTTP wrapper for all module API calls.	Adds auth, `X-Correlation-Id`, timeout, retry for safe GETs, idempotency support for commands.
`api/errors`	Normalized errors for screens.	Maps backend `{code,message,details}` to field errors, banner errors, retryable errors, and forbidden states.
`rbac/permissions`	Route/action visibility.	Uses JWT claims plus EM-CFG-03 permission catalog. Hiding a button is UX only; backend still enforces.
`i18n`	Labels, validation messages, date/number/currency.	Supports Kenya-first English/Swahili config and future regional language packs.
`files`	Upload/download wrappers.	Uses FOUNDATION_FILE_STORAGE signed URLs or upload sessions; never sends raw object-store credentials to UI code.
`telemetry`	Frontend observability.	Emits page load, API latency, failed command, offline queue size, sync outcome.

7. API Client Contract

All apps use one client wrapper:

```
type ApiRequest = {
  method: "GET" | "POST" | "PATCH" | "DELETE";
  path: string;
  body?: unknown;
  idempotencyKey?: string;
  timeoutMs?: number;
};

type ApiError = {
  status: number;
  code: string;
  message: string;
  fieldErrors?: Record<string, string>;
  retryable: boolean;
  correlationId: string;
};
```

Command calls must pass an idempotency key when the backend DD requires it. The key should be stable for one user action and should survive offline retry, for example `sales-order:{localDraftId}` or `wo-finalize:{workOrderId}:{localAttemptId}`.

8. Shared State Shape

Use the same state categories across apps:

```
type RemoteState<T> =
  | { status: "idle" }
  | { status: "loading"; staleData?: T }
  | { status: "ready"; data: T; fetchedAt: string }
  | { status: "error"; error: ApiError; staleData?: T };

type CommandState =
  | { status: "idle" }
  | { status: "submitting"; idempotencyKey: string }
  | { status: "accepted"; operationId?: string; resourceId?: string }
  | { status: "failed"; error: ApiError };
```

Every panel or card that calls a module API owns its own `RemoteState`. A failed invoice panel must not blank the Customer 360 header; a failed ticket panel must not block subscription actions.

9. Offline Queue Contract

Mobile apps use this shared queue record:

```
type OfflineCommand = {
  localId: string;
  commandType: "LEAD_CREATE" | "ORDER_SUBMIT" | "WO_FINALIZE" | "EVIDENCE_UPLOAD" | "TICKET_CREATE";
  idempotencyKey: string;
  endpoint: string;
  payload: unknown;
  attachments?: Array<{ localUri: string; filePurpose: string }>;
  createdAt: string;
  lastAttemptAt?: string;
  attemptCount: number;
  status: "PENDING" | "SYNCING" | "FAILED_RETRYABLE" | "FAILED_CONFLICT" | "SYNCED";
};
```

Only draft/pending commands may live offline. Committed business state must be confirmed by the owning module API after sync.